SE-JEPA: Self-Evolving Latent World Models

Anonymous CVPR submission

Abstract

Latent world models let agents plan by imagining the outcomes of their actions, but they stop learning once deployed: when objects or appearances shift, prediction errors become planning failures. Test-time adaptation such as AdaJEPA repairs these errors within an episode but then resets, meeting every recurring shift as if for the first time. We therefore seek a world model that remembers what it learns. However, we find that simply keeping the adapted weights fails: updates across episodes are nearly orthogonal, and accumulating them degrades planning. To overcome this, we present SE-JEPA, a self-evolving latent world model. Inspired by complementary learning systems, it pairs fast within-episode adaptation with slow cross-episode consolidation, replaying experience into slow learners, one per self-discovered domain. Supervised only by its own prediction errors, SE-JEPA decides what to consolidate, where to store it, and when to trust it for planning. Because its pretrained model is never overwritten, nothing it already knows can be forgotten. It requires no shift labels or task boundaries and benefits both gradient-based and sampling-based planners. Across unseen object shapes, visual shifts, and deformable rope and granular manipulation, SE-JEPA grows stronger the longer it is deployed, setting a new state of the art by a wide margin.

1. Introduction

A visual world model can serve many goals because it predicts the consequences of actions. Changing the goal changes which future the planner prefers, while knowledge of how the world responds to an action can carry over [2, 18, 22, 42]. During deployment, each action also gives the agent a new observation against which to check its prediction. An unexpected outcome may reveal a change in the dynamics, or in how familiar dynamics appear to the camera. Even if the agent encounters that change while pursuing one goal, the experience can help it plan when the same conditions return.

We introduce SE-JEPA, a method for learning from recurring conditions during deployment of an action-conditioned latent world model. It retains trajectories across goals, trains predictors on compatible experiences, and uses fresh observations to decide which predictor should guide the next plan. A fast update can repair a prediction during the current episode, as in AdaJEPA [35]. Carrying the update into the next episode assumes that the next trajectory needs the same correction. In our matched-goal diagnostic, direct weight inheritance loses more goals than it gains over episodic reset (Sec. 3.1). The underlying actions and their observed outcomes remain valid training examples after the goal, the plan and the fast weights have changed.

Replaying past interactions can adapt a single predic-

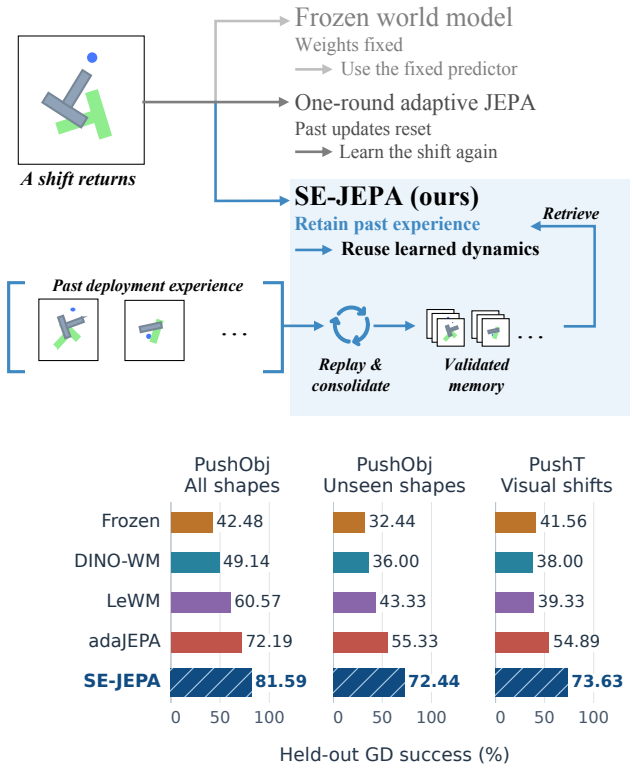


Figure 1. **Planning when a shift returns.** The schematic shows how SE-JEPA retrieves a predictor learned from earlier encounters. Bars show held-out GD success, ordered by all-shape PushObj success. One-round adaptive JEPA uses episode-local adaptation [35]. DINO-WM and LeWM use 50 targets per shape or visual condition. SE-JEPA reports the mean over three learning chains.

tor to a sustained change in dynamics [41]. When several conditions recur, the relevant experience may be separated by many unrelated episodes. Drawing on complementary learning systems [23], SE-JEPA couples fast adaptation to a slower memory organized around those recurrences. The agent retains trajectories that its pretrained predictor struggles to explain. It groups encounters into *predictive domains* using the directions of their fast updates and how well a predictor learned from one set of trajectories forecasts the next. Each domain trains a slow predictor on trajectories collected across goals. These groups emerge from actions and observations without labels identifying the task or the onset of a shift.

Before a slow predictor learns from an incoming trajectory, SE-JEPA compares its forecasts with those of the pretrained predictor on the actions actually taken. A record of better forecasts makes the slow predictor eligible for use. Early observations in a later episode then let the planner choose among eligible predictors and the pretrained reference (Fig. 1, top). The planner uses the selected predictor to score candidate actions in the usual way. This lets the model reuse experience across goals while leaving the choice of action optimizer open.

We evaluate reuse by fixing the learned memory and testing it on fresh goals. On PushObj and PushT, the full method improves held-out planning across changes in shape and appearance with task-trained ResNet models. The gains hold with gradient descent (GD, Fig. 1, bottom) and with the cross-entropy method (CEM). On both tasks, SE-JEPA reaches more goals by the midpoint of the GD replan budget than AdaJEPA does by the end (Fig. 5).

Rope and granular manipulation test the replay mechanism with a different representation and different dynamics. Using DINO-WM [42], we train one slow predictor per material by replay and use it directly for planning. Both materials finish closer to their target configurations under the same action budget. On previously unseen rope instances, a predictor frozen after one deployment pass reduces mean Chamfer distance to the goal by 47% relative to the pretrained model (Appendix B).

2. Related Work

World models for control. World models support control in several ways. Embed to Control [37], PETS [6], and PlaNet [9] predict the consequences of candidate actions for planning. World Models [8], SimPLe [16], IRIS [24], and the Dreamer family [10–12] use learned simulators to train a policy. MuZero and EfficientZero [29, 38] learn models for tree search, while TD-MPC and TD-MPC2 [13, 14] combine latent dynamics with value learning. We focus on goal-image planning, where a model can learn directly from the discrepancy between a predicted and an observed transition.

Predictive representations and JEPA. Predicting future representations is central to JEPA [18] and earlier work on predictive embeddings [30, 32]. Self-supervised objectives learn visual features by matching related views or regularizing the embedding distribution [3, 7, 40]. I-JEPA [1] predicts embeddings of masked image regions, V-JEPA [4] extends this idea to video, and V-JEPA 2 [2] conditions predictions on actions for robotic planning. For control, the encoder can draw on pretrained visual or manipulation features [5, 25, 26]. DINO-WM [42] predicts frozen patch embeddings, whereas LeWorldModel (LeWM) [22] trains its encoder and predictor together from pixels. Temporal straightening [36] shapes the latent trajectories used for planning, and DDP-WM [39] separates primary dynamics from background updates. Our method addresses how to update the predictor once the agent begins to act.

Test-time adaptation. Test-time training and entropy minimization update deployed models from unlabeled observations [31, 33], including under changing distributions [34]. In control, each executed action supplies a target for the dynamics predictor. AdaJEPA [35] uses this signal within an episode and resets the weights at its end. JEPA-TTT [41] studies a dynamics shift that persists across episodes. It trains one predictor by replaying overlapping windows from the accumulated interactions. SE-JEPA addresses recurring shifts. It keeps the pretrained reference, groups compatible trajectories, and chooses among predictors that have improved on the reference on fresh trajectories.

Continual learning and complementary memory. Complementary learning systems pair rapid acquisition with slow consolidation [23]. Continual learners replay earlier experience [27], constrain updates to protect past tasks [17, 21], preserve earlier outputs through distillation [19], or retain separate task modules [28]. For a planner, retrieving a predictor also changes the actions taken and hence the data collected next. SE-JEPA uses the first observed transitions of each episode to select among eligible predictors, including the pretrained reference.

3. SE-JEPA

3.1. Motivation

Consider two episodes in which the agent pushes the same unfamiliar object toward different goals. The first may reveal how an off-center push rotates the object. That observation can help the agent choose a push in the second episode, even when it approaches the object from a different side. An episodic learner corrects its predictions along the first trajectory [35]. Carrying its weights forward assumes that the next trajectory calls for a similar correction.

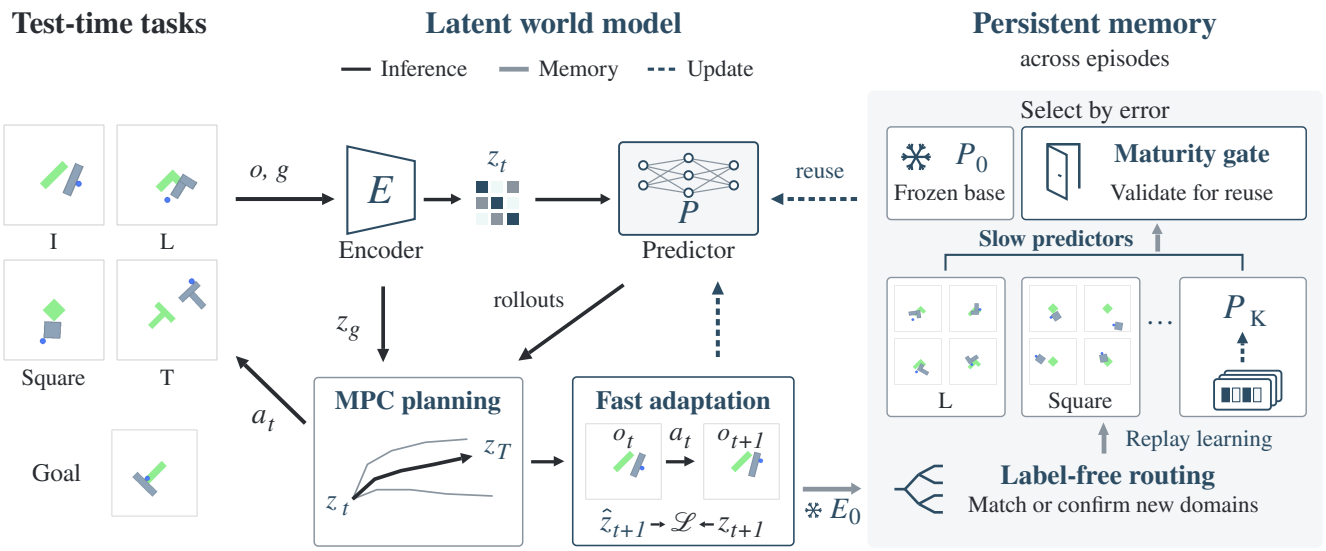


Figure 2. **SE-JEPA architecture.** The fast loop plans in latent space and adapts the predictor from observed transitions. At the end of an episode, the slow system re-encodes the trajectory with the frozen encoder, assigns unfamiliar experience to a self-discovered domain, and trains that domain's slow predictor by replay. Predictors that pass the maturity gate compete with the frozen base at the start of the next episode. Fast parameters reset at every episode; slow predictors and replay persist. Shape names illustrate memory contents and are not supervision.

We compared these choices on the same stream of 100 PushObj goals. Inheriting the previous episode's fast weights recovered 11 goals that a reset had missed, but lost 25 that the reset had solved (Table 2). In a separate seven-round stream, the mean cosine between consecutive fast-update fingerprints is 0.018 for both GD and CEM, over 2,449 pairs per planner. Absolute cosine is below 0.1 for 96.3% and 96.8% of the pairs, respectively (Appendix D). A trajectory offers a different way to retain what the agent learned. Even a push that rotated the object away from its goal tells the agent which rotation followed that contact. If a later goal calls for that rotation, the planner benefits from having a predictor that can anticipate it. The learning signal is available as soon as the next observation arrives, regardless of whether the episode eventually succeeds.

SE-JEPA keeps a fast model for the current episode and trains a slow predictor on trajectories from multiple encounters (Fig. 2). The agent receives no labels telling it when a shape or appearance recurs. It groups trajectories by how they change the model and how well a common predictor explains them. When a new goal arrives, the first few actions provide evidence for which predictor to use.

3.2. Fast adaptation and replay consolidation

Each episode begins with the pretrained encoder E_0 and predictor P_0. To plan toward a goal image, the agent encodes the current observation and the goal, rolls out candidate action sequences with the current predictor, and chooses the sequence with the lowest latent goal cost [22, 42]. It executes the first action block. Following AdaJEPA [35], it then takes one Adam step on the prediction loss over recent transitions to update the predictor's last block and the encoder head before replanning. These fast changes last for the episode. During the first few replans, the agent can also retrieve a slow predictor as a base for adaptation (Sec. 3.4).

Across episodes, SE-JEPA maintains a replay buffer $\mathcal{R}_k$ and a slow predictor P_{ϕ_k} for each group k of compatible trajectories, which we call a *predictive domain*. After an episode, the frozen encoder E_0 re-encodes the executed trajectory. This puts observations from different episodes in a common latent space even as the fast encoder changes. Once a trajectory is assigned to a domain, the agent adds it to $\mathcal{R}_k$. The slow predictor trains on windows sampled from that buffer by minimizing

$$\mathcal{L}_k(\phi_k) = \mathbb{E}_{(x,z^+)\sim\mathcal{R}_k} \left\| P_{\phi_k}(x) - z^+ \right\|_2^2. \quad (1)$$

Here x contains a short history of observation embeddings and executed actions, and z^+ is the embedding of the next observation. Each slow predictor starts as a copy of P_0. A replay update samples episodes from the buffer and draws a short window from each. The same update can therefore include a push that translated an object and an earlier push that turned it. Both windows describe observed motion in the coordinates of E_0, even if fast adaptation changed the encoder differently in the two episodes. The goal image enters when the planner uses this predictor to compare candidate futures.

3.3. Discovering predictive domains

The frozen reference first tests whether an episode contributes new experience. Let $\ell_{e,t}^{(0)}$ be its next-frame prediction error, and $\mathcal{I}_e$ index up to the first three observed transitions. Their geometric mean is

$$F_e = \exp\left(\frac{1}{|\mathcal{I}_e|}\sum_{t\in\mathcal{I}_e}\log \ell_{e,t}^{(0)}\right). \tag{2}$$

If $F_e < \tau_f$, the calibrated familiarity threshold, the agent keeps P_0 as its adaptation base and skips the episode's direct memory write. Replay thus receives episodes the reference predicts poorly.

For an unfamiliar episode, Δ_e concatenates the fast predictor and encoder-head changes. Its normalized direction and similarity to domain k are

$$u_e = \frac{\Delta_e}{\|\Delta_e\|_2}, \qquad s(e,k) = u_e^\top c_k, \tag{3}$$

where c_k is the domain's unit-length prototype. The retrieved domain is the first write candidate; otherwise, candidates are ranked by $s(e,k)$. A candidate accepts the trajectory if its predictor passes the error check in Sec. 3.4. Unassigned episodes wait in a pool. When a new direction aligns with those of two waiting episodes, the three trajectories establish a new domain.

Two contacts with the same object can induce different fast updates. Delaying domain creation gives the planner a way to keep an unusual contact until it sees a similar one again. The pool retains the founding trajectories, so learning can begin with examples from several encounters. As more evidence arrives, two domains merge if each predictor transfers to the other's recent episodes and both outperform the reference. Testing transfer in both directions checks that the proposed merger covers the experience of each group. A mature domain can also absorb a smaller group whose trajectories it already predicts well. Appendix A gives the thresholds and the rules for maintaining each prototype.

3.4. Validated retrieval for planning

A slow predictor earns a place in the planner by predicting trajectories it has yet to train on. Before adding trajectory τ_e to a buffer, the agent rolls out the candidate predictor and the pretrained reference under the actions actually taken. Their open-loop errors in the frozen latent space give the ratio

$$r_{e,k} = \frac{\mathcal{L}_{\text{roll}}(\phi_k;\tau_e)}{\mathcal{L}_{\text{roll}}(\theta_0;\tau_e)}. \tag{4}$$

Here θ_0 denotes the weights of P_0. Comparing both predictors on the same sequence of pushes makes the reference specific to the encounter. A large error on a difficult turn, for example, can still favor the slow predictor if it forecasts the turn more accurately than P_0. A large ratio rejects a write. Accepted episodes leave a record of how well the predictor transferred to a fresh trajectory, since the agent scores each one before replaying it. A domain becomes eligible for retrieval once this record shows a sustained improvement over the reference (Appendix A).

During the first three replans, the agent compares eligible predictors and P_0 on the transitions observed so far, $\mathcal{B}_{e,t}$. It selects the predictor with the lowest recent error,

$$k^\star = \operatorname*{arg\,min}_{k\in\{0\}\cup\mathcal{K}_{\text{eligible}}} \mathcal{L}_{\text{pred}}(\phi_k;\mathcal{B}_{e,t}), \qquad \phi_0 = \theta_0. \tag{5}$$

The selected weights become the base of the fast predictor, preserving the corrections already made in the current episode. For an episode the reference recognizes as familiar, the base stays at P_0. Thus past trajectories determine which predictors are eligible, and the current actions and observations determine which one guides planning.

4. Experiments

4.1. Evaluation setup

Tasks and backbones. Each episode starts with a scene and an image goal. The planner observes the consequences of its actions and receives no shift labels or task-change signals. We study shape and appearance shifts with ResNet-based world models trained on task trajectories [15]. In *PushObj*, pretraining covers T, L, Z, and plus shapes. Deployment also introduces I, square, and small tee. *PushT* keeps the T shape and changes its appearance through blur, noise, darkness, or recoloring of the agent, object, or goal. We also test replay in two deformable manipulation tasks, *rope* and *granular*, using DINO-WM with pretrained DINOv2 features [20, 26, 42].

Deployment and evaluation. For the standard PushObj and PushT comparisons, we evaluate three learned memory states per planner. Targets are resampled over five rounds, with 50 episodes per shape or visual condition in each round. We then fix each learned memory and test it on held-out goals from three new seeds. Retrieval and fast adaptation stay active. We report the fraction of goals reached and the mean number of replans, counting the full budget for failures. On rope and granular, we isolate slow consolidation by training one predictor per task with fast adaptation disabled. The planner revisits the same 50 instances over four passes. We measure Chamfer distance (CD) between the final and goal configurations and test the rope predictor separately on new instances.

Baselines and planners. Frozen (pretrained) plans with a fixed model. AdaJEPA [35] adapts within each episode.

Table 1. **Goal-conditioned planning with CEM.** Our PushObj evaluation covers seven shapes, three unseen during pretraining. PushT covers six visual shifts, including three recolorings. Success rates are percentages. Steps count MPC replans, with failures assigned 30. Section 4.1 describes the published Unseen and Chamfer distance (CD) results.

	PushObj			PushT visual shifts			Rope	Granular
Method	All (7) ↑	Unseen ↑	Steps ↓	Shift (6) ↑	Color (3) ↑	Steps ↓	CD ↓	CD ↓
DreamerV3 [12]	—	18.0	—	5.7	0.0	28.91	2.49	1.05
IRIS [24]	—	14.0	—	6.0	6.7	28.85	1.11	0.37
R3M [25]	18.9	18.0	25.49	21.7	27.3	25.00	1.13	0.95
DINO CLS [26]	14.9	10.0	26.77	34.7	35.3	22.38	0.84	0.79
DINO-WM [42]	62.0	46.0	16.15	57.0	65.3	17.32	0.491	0.301
LeWM [22]	64.0	50.0	16.28	40.3	16.0	21.45	0.786	0.322
DDP-WM [39]	57.7	43.3	17.59	52.7	57.3	18.39	0.677	0.259
Frozen (pretrained)	43.8	36.2	21.08	48.3	33.6	20.10	0.491	0.301
AdaJEPA [35]	70.4	54.4	16.02	60.2	42.2	18.46	0.629	0.304
SE-JEPA	**79.1**	**69.9**	**13.53**	**76.1**	**65.2**	**14.46**	**0.252**	**0.220**

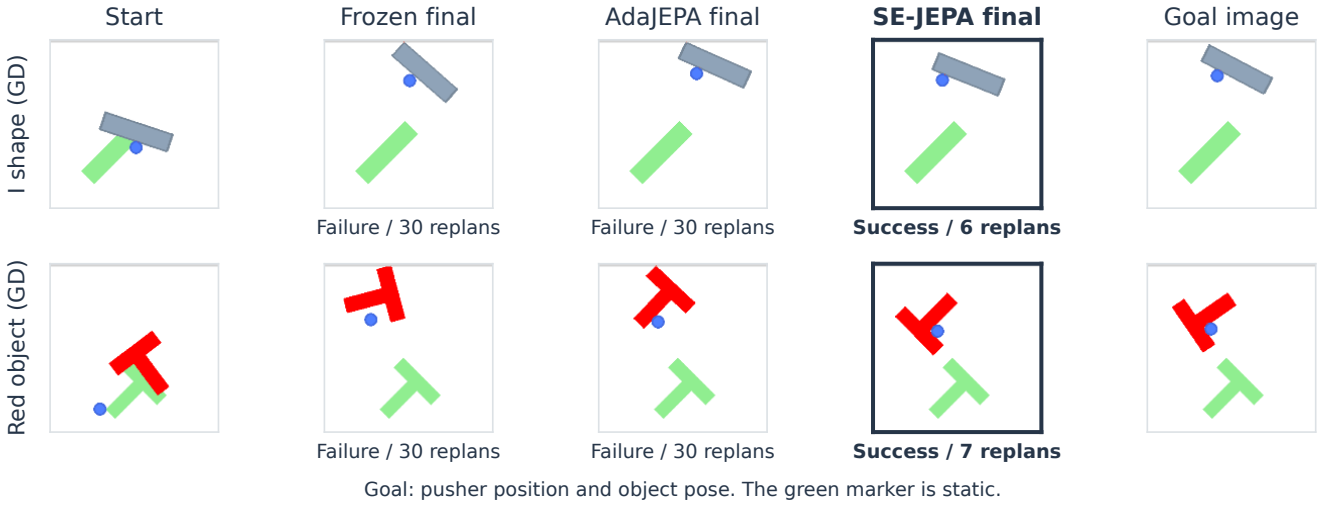


Figure 3. **Persistent memory resolves failures of episodic adaptation.** Selected GD examples on an unseen I shape (top) and red-object shift (bottom). Frozen and AdaJEPA exhaust 30 replans in both cases; **SE-JEPA** succeeds in six and seven, respectively. Each row shares the same start and goal. The goal image specifies the pusher position and object pose; the green marker is static.

On rigid objects these two baselines use AdaJEPA's official task-specific checkpoints and the same held-out targets as SE-JEPA. We evaluate both GD and CEM, executing five actions per replan up to a budget of 30 replans. Additional baselines include DINO-WM, LeWM, DDP-WM, and planners using DINOv2 CLS or R3M features [22, 25, 26, 39, 42]. DINO CLS and R3M use T-only checkpoints. On PushT, CEM uses DreamerV3 [12] and IRIS [24] as offline-trained predictors.

Table 1 takes the DreamerV3 and IRIS Unseen scores and the rope and granular CDs for DreamerV3, IRIS, R3M, and DINO CLS from Zhou et al. [42]. Their PushObj evaluation uses two unseen shapes and requires both the pusher and the object to reach their targets. Our evaluation covers three unseen shapes. Their deformable evaluation uses ten instances per task. Appendix B and Appendix C detail the protocols.

4.2. Planning with memory on held-out targets

Generalization across shapes and appearances. On shapes absent from pretraining, held-out CEM success is about 15 percentage points higher with SE-JEPA than with AdaJEPA (Table 1). Figure 7 shows how the gain varies by shape and planner. Among the three unseen shapes, both planners gain most on the I shape, followed by the square and the small tee (Table 6). Each shape recurs during deployment, but the agent must bring it to a new pose at test time. The predictor can reuse the dynamics learned from earlier pushes as the planner searches for an action sequence that reaches the new goal.

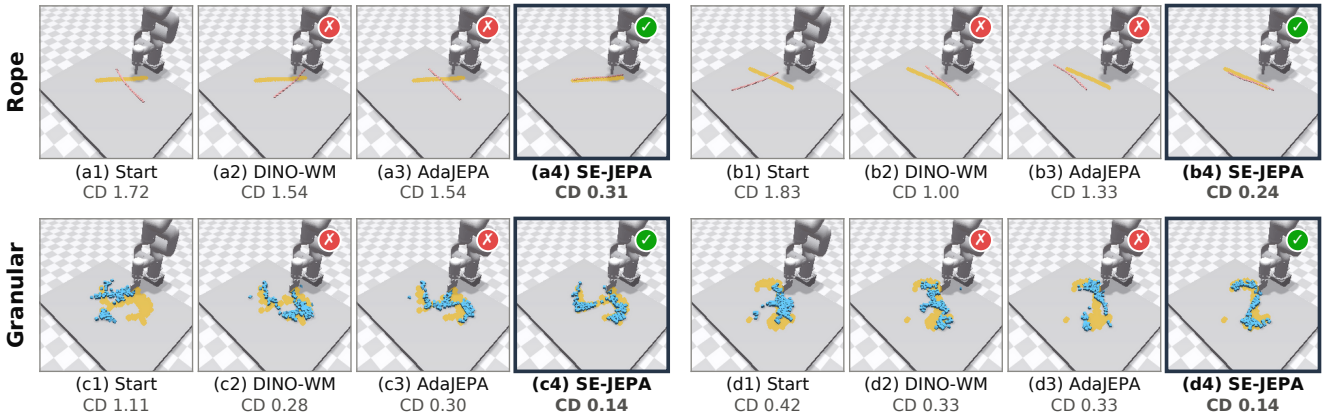


Figure 4. **Consolidated predictions support deformable manipulation.** Real FleX simulator trajectories for rope (top) and granular material (bottom), with two instances per task. Each group shows the start and the three methods' final states after 15 pushes; gold denotes the goal. These third-pass examples illustrate successful slow-only consolidation and were selected where both baselines fail. Frozen DINO-WM frames replay its executed actions. The aggregate fourth-pass results are reported in Table 1.

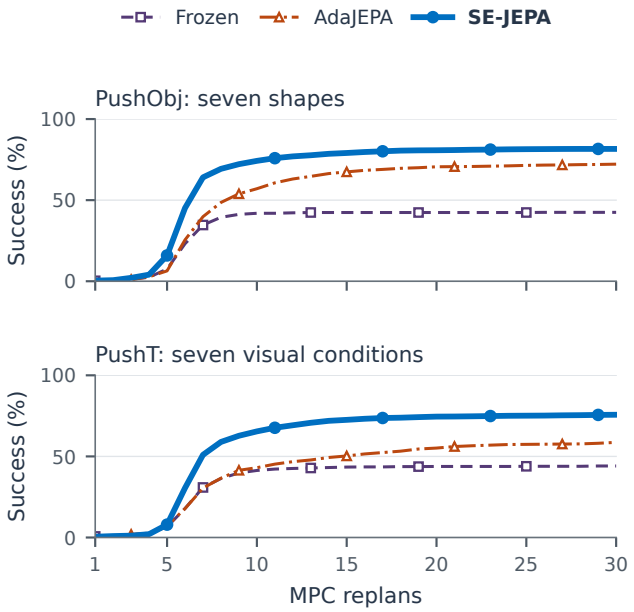


Figure 5. **Higher success within the same interaction budget.** Cumulative held-out success under GD. Curves use all seven shapes (top) and all seven visual conditions (bottom).

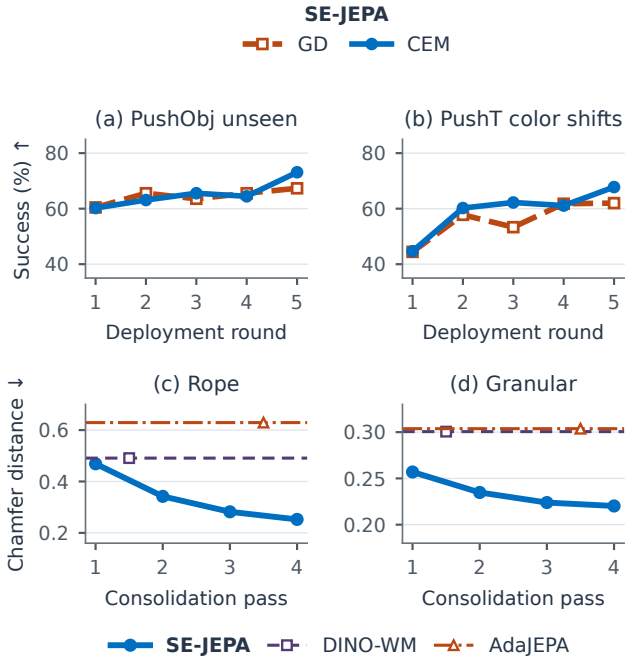


Figure 6. **Deployment experience improves two backbone families.** Top: ResNet-based models, with five rounds of online success on unseen PushObj shapes and three PushT color shifts; curves average three learning chains with targets resampled each round. Bottom: DINO-WM, with four consolidation passes over 50 fixed rope or granular instances; curves show mean CD. The horizontal lines are frozen DINO-WM and AdaJEPA.

PushT holds the object geometry fixed and changes how the scene looks. Recoloring the object provides a particularly clear comparison. With CEM, the frozen and episodically adapted models reach the same fraction of red-object goals. With memory, success rises by over thirty percentage points relative to either baseline (Table 7). The red-goal condition also shows a large gain. In both conditions the agent can draw on pushes observed in earlier scenes with that appearance, even though the current episode supplies a different target.

Planning efficiency. Figure 5 follows the full model as the replan budget increases. Halfway through the budget,

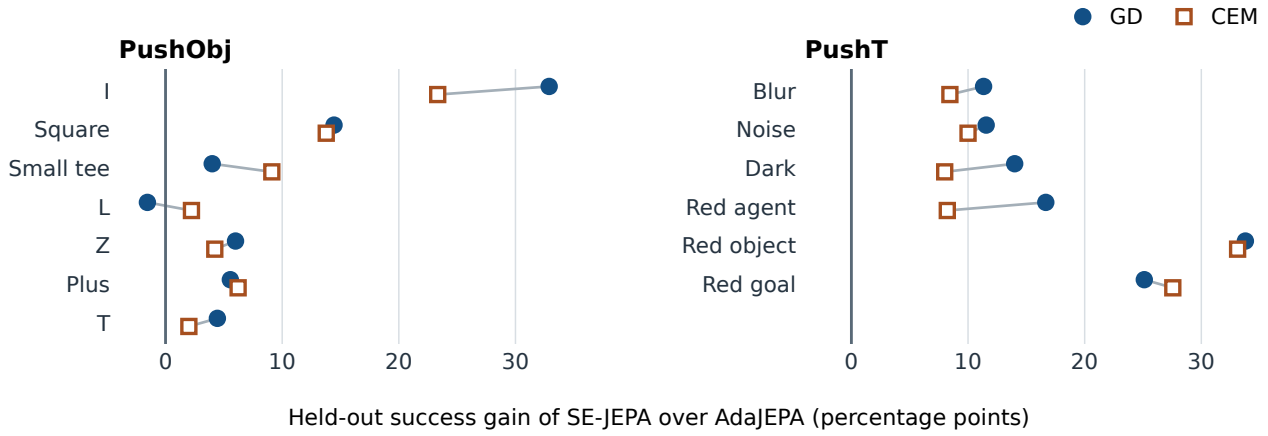


Figure 7. **Planning gains by shape and visual condition.** Each point is the difference in held-out success between full SE-JEPA and AdaJEPA for one planner, in percentage points. Positive values favor SE-JEPA. I, square and small tee are the three shapes first encountered during deployment.

SE-JEPA has already solved a larger fraction of goals than AdaJEPA reaches with the full budget. This holds on the seven PushObj shapes and on the seven PushT visual conditions, both with GD. The curves count every goal presented to each method, including those still unsolved when the budget expires. Earlier successes matter when the controller has a fixed number of pushes to reach the goal. With CEM, the same ordering appears in the mean number of replans on both tasks (Appendix B).

Consolidation on a second backbone. On rope and granular, replacing the frozen DINO-WM predictor with the consolidated predictor brings both materials closer to their target configurations (Table 1). The local deformable evaluation gives each planner the same budget of 15 pushes. A lower final CD means the planner uses those pushes to better match the rope's shape or the distribution of particles in the goal. The planner has access to one slow predictor per task and makes no fast updates during an episode. The improvement therefore tests the part of SE-JEPA that learns from previous trajectories, using DINOv2 features in place of the ResNet features used on rigid objects.

4.3. Qualitative results

Figure 3 shows two episodes in which all three methods start from the same scene and aim for the same goal. On the unseen I shape, SE-JEPA reaches a pose that both baselines miss. In the red-object episode, the baselines leave the T at the wrong orientation, while SE-JEPA turns it into the goal pose. Figure 4 shows the final rope and particle configurations after the executed pushes. With the slow predictor alone, the planner moves the rope toward its target shape and the particles toward the goal distribution. Appendix E gives additional trajectories.

4.4. Learning over deployment

When targets are resampled each round, both GD and CEM finish deployment with higher online success on unseen shapes and recolored scenes than in the first round (Fig. 6, top). These curves track performance as memory grows. Table 1 tests that memory after learning, on independent rigid-object goals.

Revisiting the same rope and granular instances reduces final CD over successive passes (Fig. 6, bottom). To test whether the rope predictor also transfers, we froze it after its first pass and used it on 91 previously unseen instances. Mean CD fell from 0.664 to 0.349, a 47% reduction relative to pretrained DINO-WM. The paired bootstrap 95% interval for the mean reduction is [0.162, 0.470] (Appendix B). The transfer test pairs the two predictors on new starting configurations and goals, with weights fixed throughout evaluation.

Selective writing and retrieval. Across seven PushObj rounds, about one episode in four adds experience to a replay buffer, and a similar fraction finishes with a slow predictor. This holds for both planners (Appendix D). An episode can contribute training data to a domain whose predictor is still awaiting validation. The pretrained model remains available on familiar encounters. On the four shapes seen in pretraining, SE-JEPA also improves held-out success over AdaJEPA with both planners (Appendix D).

4.5. Ablation study

Accumulating fast updates across episodes. Near-orthogonal update directions motivate testing whether a fixed fraction of each update transfers to the next goal. We compare $s_{e+1} = s_e + w\Delta_e$, where s_e is the starting offset from the pretrained weights and Δ_e is the episode's fast

update. Every setting receives the same 100 goals in five ordered blocks (I, square, T, I, square). Resetting between episodes reaches 62% of goals. Full inheritance reaches 48%, losing 25 reset successes and recovering 11 (Table 2, exact McNemar $p = 0.029$). The smallest tested coefficient reaches 64%, while larger coefficients progressively reduce success. Thus the degradation depends on how strongly past updates are accumulated.

Table 2. **Fast-update accumulation on 100 matched goals.** The coefficient w scales the update retained between episodes. Wins and losses are relative to episodic reset.

Accumulation w	Success (%)	Wins / losses
0 (episodic reset)	62.0	—
0.003	64.0	7 / 5
0.01	59.0	5 / 8
0.03	54.0	4 / 12
1 (naive inheritance)	48.0	11 / 25

Replay and fixed learning rates. Table 3 compares retention mechanisms on the seven-shape stream. Each variant starts from the same backbone checkpoint and is tested on the same 1,050 held-out goals. Direct weight inheritance reaches 64.67%, compared with 76.95% for episodic reset and 81.43% for the full model. A shared slow predictor trained by replay reaches 81.24%, exceeding fast-weight averaging by 5.52 points. The gap is concentrated on the unseen I shape, where replay solves 44 additional goals out of 150. The four pretraining shapes differ by one solved goal.

Table 3. **Retention mechanism and fixed slow learning rate.** GD results on the same 1,050 held-out PushObj goals, with one learned state per variant. All variants share the initial checkpoint and learning stream.

Method or slow learning rate	Success (%)	Replans
Full SE-JEPA (10^{-4})	81.43	11.37
Episodic reset	76.95	12.54
Inherit fast weights	64.67	16.12
Shared slow predictor (EMA)	75.71	12.84
Shared slow predictor (replay)	81.24	11.41
Without maturity screening	80.86	11.46
Full SE-JEPA with a fixed slow learning rate		
10^{-4} (default)	81.43	11.37
5×10^{-5}	79.62	11.79
2×10^{-4}	82.38	11.26
3×10^{-4}	80.57	11.72
5×10^{-4}	78.38	12.40
10^{-2}	76.95	12.54

The fixed-rate sweep reaches 79.62–82.38% for learning rates from 5×10^{-5} to 3×10^{-4}. At 10^{-2}, success falls to 76.95%. The deployed system uses 10^{-4}. These held-out results complement the online accumulation diagnostic above. Appendix D specifies the per-variant protocols and the visual-task learning-rate comparison.

Reading and writing on familiar episodes. The full model normally uses the pretrained predictor on familiar episodes and skips their direct replay write. We remove the read and write protections separately and together, retaining the same maturity criterion (Appendix D). With GD, removing either protection alone lowers success on this stream. Removing both reaches 83.14% with GD and 79.62% with CEM. This comparison uses one learning chain per planner.

5. Conclusion

SE-JEPA lets a visual planner reuse what it learned on earlier encounters. It keeps trajectories that the pretrained predictor struggles to explain, groups compatible experience for replay, and tests the resulting predictors on fresh trajectories before using them for control. With task-trained ResNet models, this memory improves held-out planning success and reduces replans on rigid objects under both GD and CEM. With a DINO-WM backbone, slow consolidation lowers goal error for rope and granular manipulation, and a rope predictor trained in one pass transfers to new instances.

Future work. Two objects can respond alike to an initial push and diverge at the next contact. A longer interaction history, or a short probe chosen to distinguish candidate predictors, could help route these episodes. We also observed that, late in deployment, prediction error can keep falling after planning success levels off. One way to study this gap is to test whether a more accurate predictor ranks candidate pushes in a better order near the goal. That test could guide the choice of planning horizon and how much search to spend on the final contacts.

LLM Usage Statement

Large language models (OpenAI Codex and Anthropic Claude) were used to draft and revise substantial portions of the manuscript text, and to assist with literature and source checks, LaTeX organization, figure generation, and result aggregation. They also assisted with experimental code, including offline world-model adapters and experiment orchestration. All quantitative results are taken from saved experimental outputs or explicitly cited published sources; language-model output is not used as experimental evidence. The authors take full responsibility for the claims, citations, code, and final verification of the manuscript.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 2

[2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1, 2

[3] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations*, 2022. 2

[4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024. 2

[5] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *IEEE/CVF International Conference on Computer Vision*, 2021. 2

[6] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018. 2

[7] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, 2020. 2

[8] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. 2

[9] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, pages 2555–2565, 2019. 2

[10] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020. 2

[11] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations*, 2021.

[12] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025. 2, 5

[13] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024. 2

[14] Nicklas A. Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, pages 8387–8406, 2022. 2

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 4

[16] Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H. Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, Afroz Mohiuddin, Ryan Sepassi, George Tucker, and Henryk Michalewski. Model-based reinforcement learning for atari. In *International Conference on Learning Representations*, 2020. 2

[17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 2

[18] Yann LeCun. A path towards autonomous machine intelligence. OpenReview position paper, 2022. 1, 2

[19] Zhizhong Li and Derek Hoiem. Learning without forgetting. In *European Conference on Computer Vision*, pages 614–629, 2016. 2

[20] Xingyu Lin, Yufei Wang, Jake Olkin, and David Held. SoftGym: Benchmarking deep reinforcement learning for deformable object manipulation. In *Proceedings of the 2020 Conference on Robot Learning*, pages 432–448, 2021. 4

[21] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017. 2

[22] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. 1, 2, 3, 5

[23] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. 2

[24] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023. 2, 5

[25] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In *Proceedings of the 6th Conference on Robot Learning*, pages 892–909, 2023. 2, 5

[26] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. 2, 4, 5

[27] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, 2019. 2

[28] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016. 2

[29] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. 2

[30] Max Schwarzer, Ankesh Anand, Rishab Goel, R. Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *International Conference on Learning Representations*, 2021. 2

[31] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning*, 2020. 2

[32] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 2

[33] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021. 2

[34] Qin Wang, Olga Fink, Luc Van Gool, and Dengxin Dai. Continual test-time domain adaptation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 2

[35] Ying Wang, Oumayma Bounou, Yann LeCun, and Mengye Ren. AdaJEPA: An adaptive latent world model. *arXiv preprint arXiv:2606.32026*, 2026. 1, 2, 3, 4, 5, 11

[36] Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. In *International Conference on Machine Learning*, 2026. 2

[37] Manuel Watter, Jost Tobias Springenberg, Joschka Boedecker, and Martin Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems*, 2015. 2

[38] Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. In *Advances in Neural Information Processing Systems*, 2021. 2

[39] Shicheng Yin, Kaixuan Yin, Weixing Chen, Yang Liu, Guanbin Li, and Liang Lin. DDP-WM: Disentangled dynamics prediction for efficient world models. *arXiv preprint arXiv:2602.01780*, 2026. 2, 5

[40] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stephane Deny. Barlow twins: Self-supervised learning via redundancy reduction. In *International Conference on Machine Learning*, pages 12310–12320, 2021. 2

[41] Zheyuan Zhang, Suyu Ye, Nakul Agarwal, Hossein Nourkhiz Mahjoub, Ehsan Moradi Pari, Daniel Khashabi, Tianmin Shu, and Vaishnav Tadiparthi. JEPA-TTT: Persistent test-time training of latent world models for planning under dynamics shifts. *arXiv preprint arXiv:2610.00722*, 2026. 2

[42] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning*, pages 79115–79135, 2025. 1, 2, 3, 4, 5, 13

Supplementary Material

A. Implementation and Memory Management

Compute environment. Model training and evaluation were conducted on systems equipped with 8× NVIDIA GeForce RTX 5090 GPUs and 8× NVIDIA A100 GPUs.

A.1. Fast and slow learning

The label-free rigid-object configuration builds on the episodic adaptation loop of AdaJEPA [35]. At every replan, the most recent observed transitions provide a self-supervised next-latent loss. The predictor's last block and the encoder's designated projection/head parameters are adapted with one Adam step. The episode-local predictor and encoder changes are reset at the next episode. A retrieved slow predictor supplies the predictor initialization; it does not replace the frozen reference encoder.

Persistent replay is re-encoded by the reference encoder. Each slow learner is a full predictor copy, initialized from the pretrained predictor rather than from the final fast parameters of its founding episode. For each slow optimization step, up to eight stored episodes are sampled with replacement; each contributes a random window of up to six latent frames. The prediction objective averages one-step observation-feature errors over the available history windows. Action features condition prediction but are not reconstruction targets. Standard writes use 15 Adam steps. Founding a learner after n stored episodes uses $15n$ steps to process the accumulated evidence. Merging likewise adds replay updates for absorbed experience.

Replay stores compact latent trajectories rather than images. The pending pool is bounded, and merging and absorption remove redundant predictor copies as compatible domains are recognized.

A.2. Order of decisions

Algorithm 1 summarizes a learning episode. Familiarity uses the geometric mean of newest-frame prediction errors on up to the first three observed transitions. The threshold is calibrated once on the shape backbone and transfers unchanged to the visual experiments.

The selected predictor's error is evaluated before it learns from the current episode, so maturity reflects transfer to data the predictor has not fitted. In read-only evaluation, the episode-end write, validation-history update, slow training, merging, and absorption steps are all disabled.

A.3. Fingerprints and delayed domain founding

A fingerprint concatenates the adapted predictor and encoder-head changes and is normalized to unit length. Domain prototypes accumulate normalized fingerprints from episodes planned from the reference; episodes starting from a slow predictor do not redefine that reference-based prototype. If an episode has already read a learner, it preferentially writes to that learner's domain unless the prospective error ratio rejects the write. Otherwise, it tries compatible domain directions in descending cosine order.

An unassigned episode enters a pending pool. It confirms a new domain when at least two pending fingerprints align with its own above the new-domain threshold. These members need not be pairwise aligned with one another; the incoming fingerprint is the common reference for this confirmation. The members and the new episode are transferred together to the new replay. No ground-truth shape or appearance category participates in this procedure.

Table 4. Label-free configuration used for PushObj and PushT. The seven-round stream in Appendix D uses a slow learning rate of 2×10^{-4}.

Setting	Value
Fast predictor learning rate	5×10^{-4}
Fast encoder-head learning rate	10^{-5}
Fast steps per replan	1
Recent adaptation buffer	5 transitions
Slow learning rate	10^{-4}
Slow steps per standard write	15
Slow batch size (episodes)	8
Slow sampled window (frames)	up to 6
Familiarity threshold τ_f	0.0146
Existing-domain cosine threshold	0.10
New-domain cosine threshold	0.05
Episodes to confirm a domain	3
Pending-pool capacity	40
Minimum episodes to train a learner	3
Retrieval window	first 3 replans
Maturity history length m	3
Maturity error-ratio threshold τ_m	0.9
Write rejection ratio	1.2
Merge evidence per direction	2 episodes
Merge relative tolerance	0.5
Fragment absorption interval	25 episodes
Absorption error-ratio threshold	0.8
Fragment-size limit	< 6 episodes

A.4. Merging and fragment absorption

For a new unfamiliar episode, every existing trained learner is evaluated before training. The resulting ratios provide cross-domain evidence. Let $G_{A\to B}$ be the geometric mean error ratio of learner B on recent episodes assigned to A, and $G_{A\to A}$ the corresponding ratio for learner A. Domains merge only if both directions satisfy

$$G_{A\to B} \le \min\{1, (1+\epsilon_m)G_{A\to A}\}, \tag{6}$$

Algorithm 1 One deployment-learning episode of SE-JEPA

Require: Frozen (E_0, P_0), memory $\mathcal{M}$, initial observation o, goal g
Reset fast parameters to (E_0, P_0)
Initialize recent buffer $\mathcal{B}$ and episode trajectory τ
while budget remains and task is not complete **do**
    Optimize an action sequence with the current fast model
    Execute its first block; observe the next state
    Append real transitions to $\mathcal{B}$ and τ
    if within the early retrieval window **then**
        Form candidates: P_0 and mature slow predictors
        Restrict to P_0 if the observed episode is familiar
        Select the smallest observed prediction error
        Load the selected predictor, retaining the fast offset
    end if
    Adapt the fast model on recent real transitions
end while
Encode τ with frozen E_0
Measure pre-update rollout errors of existing predictors
if episode is unfamiliar **then**
    Propose a domain using retrieval or an update fingerprint
    Reject an incompatible write, or retain pending evidence
    if a domain is assigned or newly confirmed **then**
        Record prospective validation when a learner exists
        Append latent experience; update its slow predictor
        Check bidirectional merges and periodic absorption
    end if
end if
Discard episode-local fast parameters

with the symmetric condition for $B \to A$. Each direction requires two jointly evaluated episodes in the primary configuration. The larger domain retains its learner and incorporates the union of stored experience. Merged identifiers are not reused.

Every 25 episodes, small immature domains can be absorbed into a mature predictor that explains their experience with an error ratio at most 0.8. The primary configuration limits absorption to fragments with fewer than six episodes and fewer than three maturity measurements. Pending-pool absorption and the additional absolute-error gate are disabled in the shape and visual experiments.

B. Evaluation Protocols

B.1. Shape shifts

The shape learning stream has five deployment rounds over seven shapes, with 50 episodes per shape in every round. The held-out benchmark uses target seeds 2000, 2100, and 2200, each with 350 episodes. I, square, and small tee are unseen in pretraining; T, L, Z, and plus are seen.

For the paired planner comparison in Table 5, each planner has three learned memory states. Every state is evaluated on all three held-out seeds. For each seed we average over episodes and states, then compute the standard deviation across seed-level means. The CEM states are S7t3, S7t3b, and S7t3s700. The SE-JEPA PushObj entries in Table 1 use these CEM states. Figure 1 uses the paired GD results for Frozen, AdaJEPA, and SE-JEPA. The local external PushObj baselines use seed 2000. AdaJEPA uses its official released PushObj checkpoint for both planners.

B.2. Visual shifts

The paired visual evaluation contains the original T condition, blur, salt-and-pepper noise, darkness, red agent, red object, and red goal. Three five-round learning chains resample targets every round, with 350 episodes per round. Held-out testing uses three target seeds and 50 episodes per condition. Direct checks of source-episode identifiers confirm that held-out sources never occur in the learning streams and that the three test seeds use disjoint sources.

The color-learning curve in Fig. 6 aggregates red-agent, red-object, and red-goal episodes. For each round and learning chain, it sums successes over the three conditions and divides by 150 episodes, and plots the mean of the three chain-level rates. The archived statistics include their sample standard deviation. The three conditions share the same 50 source targets within a chain and round.

The external visual baselines use the established VIS50 collection with planning seed 100. The external comparisons use its six shifted conditions, with 50 paired initial/goal states per condition.

For the external PushT GD bars in Fig. 1, DINO-WM and LeWM share 50 targets per condition at seed 100. Shift averages the six altered conditions, excluding the original T. Both use 100 GD steps and a 150-physical-step cap. DINO-WM executes 25 physical steps per replan (at most six replans); LeWM executes five (at most 30). Their checkpoints, training budgets and controllers differ, so these are method-level comparisons. AdaJEPA in the paired GD/CEM comparisons uses its official released visual-shift checkpoint.

B.3. Success and interaction counts

For the paired rigid-object comparisons, GD uses 100 optimization steps; CEM uses 10 iterations with 200 samples

and 30 elites per iteration. Rigid-object success requires a joint Euclidean error below 20 simulator units across the pusher and object positions, and an object orientation error below $\pi/9$. Each executed model action contains five physical actions, and an episode has at most 30 MPC replans. For episode i with success indicator S_i and first successful replan T_i,

$$\overline{T} = \frac{1}{N}\sum_{i=1}^{N}\left[\, S_i T_i + (1 - S_i)\, T_{\max}\right], \qquad T_{\max} = 30. \quad (7)$$

Equivalently, $\overline{T} = T_{\max} - \sum_{t=1}^{T_{\max}-1} F(t)$, where $F(t)$ is the fraction of episodes solved by replan t; the cumulative curves in Fig. 5 visualize F.

B.4. Deformable manipulation

Rope and granular each use one slow predictor, a DINO-WM backbone, and 50 instances revisited over four passes, with the same targets, order, and planning seeds in every pass. The slow-only variant plans from the slow predictor without fast within-episode updates. CEM uses 300 samples, 30 elites, and 10 iterations, with three MPC iterations of five pushes each. The unseen-rope evaluation freezes the predictor saved after the first pass and tests it on 91 instances absent from the learning stream, against the frozen reference on the same instances. Mean CD falls from 0.664 with the reference to 0.349 with the one-pass predictor. Across the 91 matched instances, the predictor improves CD on 56. A paired percentile bootstrap over instances (20,000 resamples, seed 20261107) gives a 95% interval of [0.162, 0.470] for the mean CD reduction. The observed mean reduction is 0.314. Two frozen granular episodes with unstable batched particle simulation were rerun individually with identical settings and seeds.

C. Baseline Checkpoints and Comparisons

Predictive world models. The comparisons focus on latent world models used for visual planning: DINO-WM, LeWM, and DDP-WM, with DINO CLS and R3M as representation baselines. For PushObj and PushT, Frozen and AdaJEPA use the official checkpoints released by AdaJEPA, rather than our trained backbones. For PushObj and PushT, SE-JEPA uses ResNet-based world models trained directly on task trajectories. Its PushObj backbone is trained for three epochs on 16,000 trajectories covering T, L, Z, and plus. The additional DINO-WM shape model uses the same training trajectory count and epoch budget, with 4,000 validation trajectories and batch size 32. DINO CLS and R3M use existing T-only checkpoints; LeWM and DDP-WM use their corresponding checkpoint adapters.

The controlled ablation in Table 3 instead starts every variant from our common pretrained checkpoint. Its episodic-reset row isolates memory while holding the backbone fixed; it is separate from the official-checkpoint AdaJEPA benchmark.

Offline generative baselines. For PushT we use offline-trained DreamerV3 and IRIS predictors for CEM planning on the same VIS50 targets as DINO-WM. The Unseen column of Table 1 includes the published PushObj results of Zhou et al. [42] for DreamerV3 (18%) and IRIS (14%, their Table 4). They train on four shapes and test on two unseen shapes, requiring both the pusher and the object to reach their targets. The local results use the seven-shape benchmark in Appendix B.

Evaluation groups. Frozen (pretrained), AdaJEPA, and SE-JEPA use the paired held-out targets in Appendix B. The external visual baselines share the VIS50 targets, with 50 initial/goal pairs in each of six shifted conditions and planning seed 100. Methods within each evaluation group receive the same targets and physical interaction budget. PushT Shift averages blur, noise, dark, red agent, red object, and red goal; Color averages the last three. The original condition contributes to the seven-condition tables and curves.

Deformable results. The rope and granular experiments use DINO-WM with pretrained DINOv2 patch features as the frozen backbone. This complements the task-trained ResNet setting used for rigid-object manipulation. AdaJEPA adapts within each episode, whereas SE-JEPA uses a consolidated slow predictor. The CD entries for DreamerV3, IRIS, R3M, and DINO CLS come from the ten-instance evaluation of Zhou et al. [42]. Our frozen DINO-WM reference uses the same 50 instances as our deformable adaptation runs and appears in both the DINO-WM and Frozen rows. The repeated-instance and unseen-rope protocols are specified in Appendix B.

D. Additional Results

D.1. All object shapes

Table 6 reports every held-out shape. SE-JEPA gives the largest gains on the unseen shapes: with GD, I rises from 44.7% (AdaJEPA) to 77.6% and square from 46.7% to 61.1%. It also leads on the seen shapes in aggregate.

D.2. All visual conditions

Table 7 and Fig. 7 break the paired visual results down by condition. SE-JEPA improves on all seven conditions with both planners. The largest gains, more than 25 points, appear under red-object and red-goal recoloring, where episodic adaptation has to rediscover the new appearance in every episode.

Table 5. Both planners on the paired held-out sets. Shape All and Unseen use held-out PushObj targets. Visual All averages seven conditions and Shift six; visual Steps averages all seven conditions.

		PushObj			PushT		
Planner	Method	All ↑	Unseen ↑	Steps ↓	All ↑	Shift ↑	Steps ↓
GD	Frozen	42.48	32.44	20.02	44.10	41.56	19.94
GD	AdaJEPA	72.19	55.33	14.52	58.67	54.89	17.96
GD	SE-JEPA	81.59	72.44	11.30	75.68	73.63	13.21
CEM	Frozen	43.81	36.22	21.08	50.86	48.33	19.56
CEM	AdaJEPA	70.38	54.44	16.02	62.95	60.22	17.87
CEM	SE-JEPA	79.08	69.85	13.53	77.05	76.11	14.15

Table 6. Held-out success (%) per PushObj shape. I, square, and small tee are unseen in pretraining. All averages the seven shapes.

Planner	Method	I	L	Square	Z	Small tee	Plus	T	All
CEM	Frozen	32.7	52.7	23.3	41.3	52.7	43.3	60.7	43.8
CEM	AdaJEPA	49.3	82.0	44.7	82.0	69.3	79.3	86.0	70.4
CEM	SE-JEPA	72.7	84.2	58.4	86.2	78.4	85.6	88.0	79.1
GD	Frozen	24.0	52.7	28.0	40.0	45.3	48.7	58.7	42.5
GD	AdaJEPA	44.7	90.0	46.7	82.0	74.7	82.0	85.3	72.2
GD	SE-JEPA	77.6	88.4	61.1	88.0	78.7	87.6	89.8	81.6

D.3. Planning efficiency

Figure 5 plots cumulative held-out success against the number of MPC replans. SE-JEPA pulls ahead within the first four replans and keeps the lead for the rest of the budget, showing how the advantage develops before the final 30-replan evaluation.

D.4. Deformable manipulation

Figure 6 shows the four-pass learning curves in the main paper. Table 8 adds standard errors to the deformable results of Table 1. Table 9 follows consolidation pass by pass at two slow learning rates. With 10^{-4}, CD decreases at every pass on both tasks. Rope improves faster in the early passes at 2×10^{-4}; the two learning rates give similar fourth-pass CD on both tasks.

D.5. Component protocols and learning rates

We assign each variant the same initial checkpoint and the same sequence of shapes and goals. The comparison uses the first learning chain for direct weight inheritance and the corresponding chain for the full model. The episodic-reset variant starts each goal from the common checkpoint. The shared replay variant trains a single slow predictor on stored trajectories. The EMA variant uses an exponential moving average of the fast weights to update a single slow predictor.

Table 3 reports the fixed-rate sweep.

For GD, disabling only the read protection produced identical held-out rollouts to episodic reset from the same checkpoint across all 21 test batches. Removing maturity screening instead allows a slow predictor to compete for retrieval earlier. This differs from the familiarity ablations in Table 10, which change how the agent handles episodes the pretrained model already predicts well. For context, a second CEM run of the full model on the same learning stream reached 80.19%, compared with 78.57% in the original run.

On PushT, doubling the slow learning rate from 10^{-4} to 2×10^{-4} changes success by $+0.19$ percentage points with GD and -1.05 with CEM. These comparisons average three learned states per planner over all seven visual conditions, including the original appearance.

D.6. Transferring fast weights across goals

We tested whether the parameter update from one episode helps on the next. Let s_e be the offset from the pretrained weights at the start of episode e, and Δ_e the fast update in that episode. We set $s_{e+1} = s_e + w\Delta_e$, where $w = 0$ resets to the pretrained model and $w = 1$ retains the full update. Each variant sees the same 100 PushObj goals in five blocks of 20 (I, square, T, I, square). Keeping the full update loses more goals than it gains relative to reset (Table 2, two-sided exact McNemar $p = 0.029$). Smaller weights give success rates closer to reset.

D.7. A seven-round shape stream

A second PushObj stream runs seven rounds with a slow learning rate of 2×10^{-4}, one chain per planner. Each round has 50 episodes per shape, drawn without replacement and excluding the held-out sources, for 2,450 episodes per plan-

Table 7. Held-out visual success (%) per condition on the paired targets. All averages seven conditions; Shift excludes Original.

Planner	Method	Original	Blur	Noise	Dark	Red agent	Red object	Red goal	All	Shift
CEM	Frozen	66.0	56.0	62.7	70.7	50.0	14.0	36.7	50.9	48.3
CEM	AdaJEPA	79.3	78.0	76.7	80.0	71.3	14.0	41.3	63.0	60.2
CEM	SE-JEPA	82.7	86.4	86.7	88.0	79.6	47.1	68.9	77.0	76.1
GD	Frozen	59.3	48.7	55.3	58.7	50.7	6.7	29.3	44.1	41.6
GD	AdaJEPA	81.3	77.3	73.3	73.3	58.0	10.7	36.7	58.7	54.9
GD	SE-JEPA	88.0	88.7	84.9	87.3	74.7	44.4	61.8	75.7	73.6

Table 8. Deformable manipulation: mean CD $\pm$ standard error over 50 instances. SE-JEPA uses its slow predictor after four passes.

Method	Rope CD ↓	Granular CD ↓
DINO-WM	0.491 ± 0.059	0.301 ± 0.020
LeWM	0.786 ± 0.112	0.322 ± 0.018
DDP-WM	0.677 ± 0.086	0.259 ± 0.012
AdaJEPA	0.629 ± 0.096	0.304 ± 0.019
SE-JEPA	$\mathbf{0.252 \pm 0.045}$	$\mathbf{0.220 \pm 0.012}$

Table 9. CD of SE-JEPA after each consolidation pass at two slow learning rates (mean $\pm$ standard error, 50 instances).

	lr 10^{-4}		lr 2×10^{-4}	
Pass	Rope	Granular	Rope	Granular
1	0.469 ± 0.100	0.257 ± 0.014	0.416 ± 0.070	0.260 ± 0.012
2	0.342 ± 0.054	0.235 ± 0.012	0.286 ± 0.050	0.241 ± 0.012
3	0.282 ± 0.033	0.224 ± 0.010	0.283 ± 0.037	0.247 ± 0.012
4	0.252 ± 0.045	0.220 ± 0.012	0.243 ± 0.033	0.243 ± 0.013

Table 10. Effect of familiarity protection on PushObj success (%). Each run uses the same five-round learning stream and 1,050 held-out goals across seven shapes. Dashes indicate configurations with no reported test.

Familiarity protection	GD	CEM
Full SE-JEPA	81.43	78.57
Disable read protection only	76.95	—
Disable write protection only	79.43	—
Disable both protections	83.14	79.62

ner. Online success rises from 70.6% in the first round to 78.6% in the seventh with GD, and from 72.3% to 78.3% with CEM.

The episode logs record the fast-update fingerprint of every episode. Figure 8 shows the cosine between consecutive fingerprints: the mean is 0.018 for both planners. Absolute cosine is below 0.1 for 96.3% of GD pairs and 96.8% of CEM pairs. These are descriptive statistics from one sequential stream per planner. These measurements support consolidating transition data rather than accumulating episode-specific parameter updates (Sec. 3.1).

D.8. Memory use across episodes

SE-JEPA learns selectively. In a seven-round PushObj stream of 2,450 episodes per planner, it writes only 25% (GD) to 27% (CEM) of episodes to memory; the rest are familiar to the pretrained model, still pending, or rejected by validation. Retrieval is just as targeted: slow predictors plan about one episode in four, and the reference handles the rest.

Selective writing leaves familiar dynamics to the pretrained model, which SE-JEPA never overwrites. On the four pretraining shapes, SE-JEPA reaches 86.0% held-out success with CEM and 88.4% with GD, against 82.3% and 84.8% for AdaJEPA. This preserves a useful pretrained reference while allowing specialized predictors to add capacity for recurring shifts.

E. Qualitative Examples and Reproducibility

E.1. Full trajectories

Figure 3 selects an unseen-I and a red-object episode where both baselines fail and SE-JEPA succeeds under GD. These illustrate changes in planning behavior; aggregate success is evaluated on the complete held-out sets. The source package supplies the first held-out target of test seed 2000 for each condition, and the displayed conditions are chosen by outcome. All methods share identical initial and goal observations. The panels use a consistent crop size to enlarge the movable objects, with every colored foreground element preserved.

Figures 9 and 10 provide additional CEM examples on the first held-out target of the same test seed: an unseen square and a red-agent shift. They show the complete trajectories and each method's prediction loss. These are separate from the GD examples in the main paper.

Each loss curve is measured in the latent space of its own model, so it describes how prediction error evolves within a method over the episode. Task success, judged in the shared simulator state, is the common yardstick across methods.

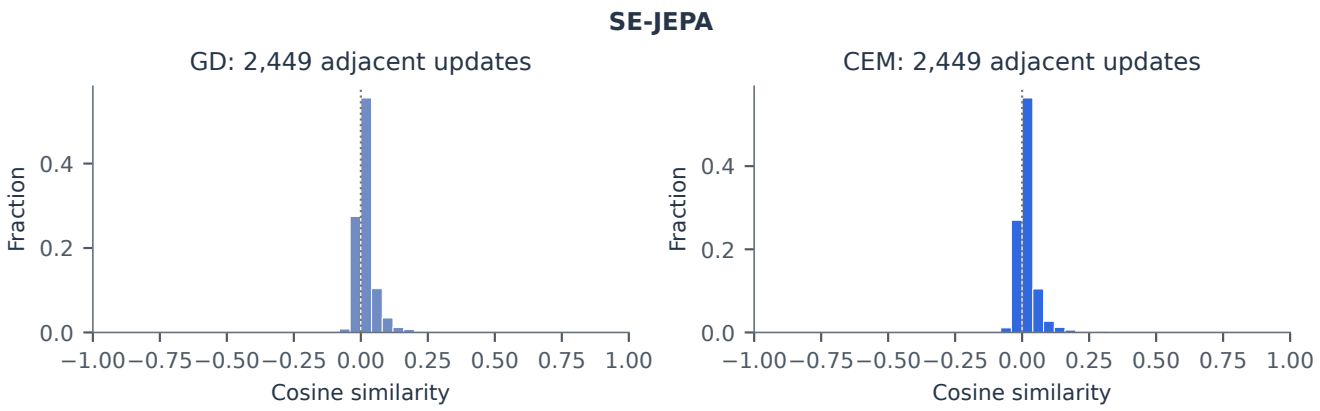


Figure 8. Cosine similarity between consecutive fast-update fingerprints in the seven-round shape stream (2,449 pairs per planner).

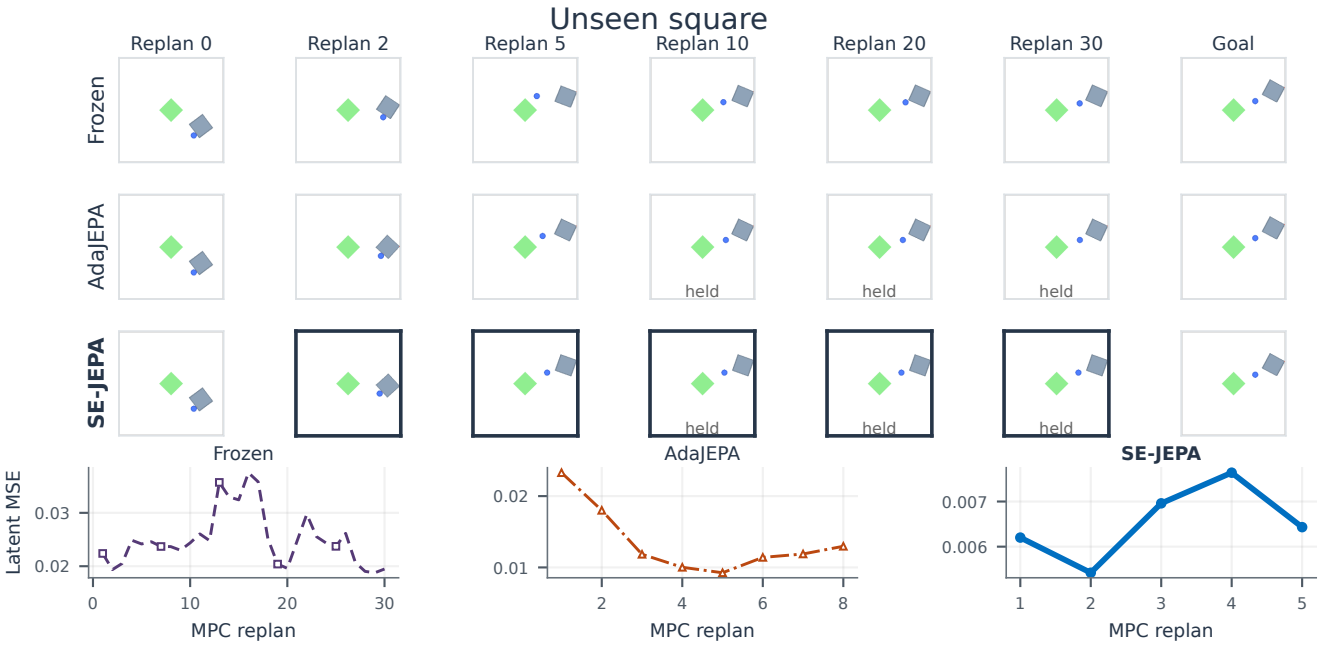


Figure 9. Full unseen-square example with intermediate frames and prediction-loss traces. Frames marked held repeat the terminal state after success. **SE-JEPA** reaches the goal at replan five, AdaJEPA at replan eight, and the frozen model does not succeed within the budget.

E.2. Reproducibility

Tables and plots use the bundled reporting snapshot and the archived Drive experiment packages. The revised trajectory panels are rendered directly from the packages' simulator frames; the asset manifest records their source files and hashes. Success and Steps are computed from the final episode files and their run configurations. Offline world-model checkpoints record the upstream code revision, the epoch budget, the model configuration, and hashes of the adapter sources. Code, configurations, and learned memory states will be released.



Figure 10. Full red-agent example, replayed in the original simulator from the executed actions. **SE-JEPA** succeeds at replan ten; Frozen and AdaJEPA reach the 30-replan limit.